

模組 11：從 Google App Engine 遷移至 Cloud Functions

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-11-functions?hl=zh-tw>

步驟數：8

瞭解如何將簡易的 Python App Engine 應用程式 (或是將大型的單體式應用程式拆分為微服務) 並移至 Cloud Functions

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 移除設定檔](#)
5. [5. 修改應用程式檔案](#)
6. [6. 建構及部署](#)
7. [7. 摘要/清除](#)
8. [8. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

有時您不需要「完整應用程式」，因此不需要 [App Engine](#) 或 [Cloud Run](#) 的資源。如果程式碼只包含微服務或簡單函式，[Cloud Functions](#) 可能更適合。本程式碼研究室會說明如何遷移簡單的 App Engine 應用程式 (或將較大的應用程式拆分成多個微服務)，並將其部署至 Cloud Functions。Cloud Functions 是專為這類用途打造的無伺服器平台。

在接下來的研究室中

- 使用 Cloud Shell
- 啟用 Google Cloud Translation API
- 驗證 API 要求
- 將小型 App Engine 應用程式轉換為在 Cloud Functions 上執行
- 將程式碼部署至 Cloud Functions

軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 GCP 帳單帳戶
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發及部署 App Engine 應用程式](#)的基本知識
- 可正常運作的[第 2 課 Cloud NDB Python 3 App Engine 應用程式](#)
- 建議：完成[第 2 模組程式碼研究室](#)，並完成將應用程式從 Python 2 移植到 3 的加分步驟

問卷調查

2. 背景

Google App Engine 和 Cloud Functions 等 PaaS 系統為使用者帶來許多便利。有了這些無伺服器平台，技術團隊就能專心打造業務解決方案，不必花時間研究要使用的平台，也不必決定所需的硬體數量。應用程式可視需要自動擴充，並以用量計費模式縮減至零，以控管費用，且支援現今各種常見的開發語言。

不過，雖然 App Engine 非常適合用於全端網頁應用程式開發或行動應用程式的複雜後端，但開發人員通常只是想將某些功能上線，例如更新新聞動態或提取主隊季後賽的最新比分。雖然這兩種情境都有編碼邏輯，但似乎都不屬於需要 App Engine 支援的完整「應用程式」。這時 Cloud Functions 就能派上用場。

Cloud Functions 適用於部署一小段程式碼，這些程式碼：

- 不是整個應用程式的一部分
- 不需要整個開發堆疊
- 位於專注於單一用途的應用程式或單一行動應用程式後端

您也可以使用 Cloud Functions 將大型單體應用程式拆分成多個微服務，每個微服務都使用共用的通用資料庫，例如 [Cloud Firestore](#) 或 [Cloud SQL](#)。此外，您也可以將函式或微服務容器化，並在 Cloud Run 上以無伺服器方式執行。

我們在幾乎所有遷移教學課程中介紹的範例 App Engine 應用程式，是具有基本功能的簡短應用程式，在 Cloud Functions 中也能正常運作。在本教學課程中，您將瞭解如何修改該應用程式，使其在 Cloud Functions 上執行。從 App Engine 的角度來看，函式比整個應用程式簡單，因此入門體驗應該會更輕鬆 (也更快)，而且一般來說「額外負擔」較少。這項遷移作業包含下列步驟：

- 設定/準備工作
 - 移除設定檔
 - 修改應用程式檔案
-

3. 設定/準備工作

本程式碼研究室會從 [Module 2 Cloud NDB App Engine 範例應用程式](#) 的 Python 3 版本開始，因為 Cloud Functions 不支援 Python 2。首先，請設定專案、取得程式碼，然後部署基準應用程式，確認我們是從可運作的程式碼開始。

1. 設定專案

如果您已完成 [第 2 模組](#) 程式碼研究室 (並移植到 Python 3)，建議重複使用該專案 (和程式碼)。或者，您也可以 [建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用 App Engine 服務，並具備有效的帳單帳戶。

2. 取得基準範例應用程式

本程式碼研究室的先決條件之一，是必須有可運作的第 2 課範例應用程式。如果沒有，請先完成上述任一教學課程，再繼續進行本程式碼研究室。如果您已熟悉內容，可以先抓取下方的模組 2 程式碼。

無論您使用自己的程式碼或我們的程式碼，我們都將從第 2 模組的 Python 3 程式碼開始。本程式碼研究室會逐步說明每個步驟，最後的程式碼會與第 11 模組存放區資料夾 (FINISH) 中的程式碼類似。

- 開始：[第 2 單元程式碼](#) (3.x [Module 2b repo folder])
- 完成：[第 11 堂課程式碼](#) (3.x)
- [整個存放區](#) (複製或下載 ZIP 檔案)

Python 3 模組 2 STARTING 檔案的目錄 (您的或我們的) 應如下所示：

```
$ ls
README.md          main.py            templates
app.yaml           requirements.txt
```

3. (重新) 部署基準應用程式

請立即執行剩餘的準備步驟：

1. 重新熟悉 `gcloud` 指令列工具
2. 使用 `gcloud app deploy` 重新部署範例應用程式
3. 確認應用程式在 App Engine 上順利執行

成功執行這些步驟後，即可將其轉換為 Cloud 函式。

步驟 4 · 約 1 分鐘

4. 移除設定檔

`app.yaml` 檔案是 Cloud Functions 不會使用的 App Engine 構件，因此請**立即刪除**。如果您沒有或忘記執行這項操作，也不會造成任何損害，因為 Cloud Functions 不會使用這項設定。這是唯一的設定變更，因為 `requirements.txt` 與模組 2 中的內容完全相同。

如果您也將 Python 2 App Engine 應用程式移植到 Python 3，請刪除 `appengine_config.py` 和 `lib` 資料夾 (如有)。這些是 Python 3 執行階段未使用的 App Engine 構件。

5. 修改應用程式檔案

只有一個應用程式檔案 `main.py`，因此所有必要的變更都會在這個檔案中進行，以便遷移至 Cloud Functions。

匯入

由於我們只處理函式，因此不需要網頁應用程式架構。不過，為方便起見，呼叫以 Python 為基礎的 Cloud Functions 時，系統會自動傳遞要求物件，供程式碼視需要使用。(Cloud Functions 團隊選取這個物件，傳遞至函式做為 [Flask 要求物件](#))。

由於網頁架構不屬於 Cloud Functions 領域，因此除非應用程式使用其他 Flask 功能，否則不會從 Flask 匯入任何內容。我們的案例確實如此，因為轉換為函式後，範本的算繪作業仍會繼續進行，因此仍需呼叫

`flask.render_template()`，並從 Flask 匯入。由於沒有網頁架構，因此不需要例項化 Flask 應用程式，請刪除 `app = Flask(__name__)`。套用變更前後的程式碼如下所示：

BEFORE:

```
from flask import Flask, render_template, request
from google.cloud import ndb

app = Flask(__name__)
ds_client = ndb.Client()
```

修改後：

```
from flask import render_template
from google.cloud import ndb

ds_client = ndb.Client()
```

如果您依附於應用程式物件 (`app`) 或任何其他網頁架構基礎架構，就必須解決所有這些依附元件、找出適當的解決方法，或是完全移除這些依附元件的使用，或尋找 Proxy。這樣才能將程式碼轉換為 Cloud Function。否則，建議您繼續使用 App Engine，或將應用程式容器化，以便在 Cloud Run 中執行

更新主要處理常式函式簽章

函式簽章必須進行下列變更：

1. 將應用程式轉換為 Cloud Function 後，就不再使用 Flask，因此請移除路徑裝飾器。
2. Cloud Functions 會自動傳入 Flask `Request` 物件做為參數，因此請為該物件建立變數。在我們的範例應用程式中，我們會將其命名為 `request`。
3. 部署的 Cloud Functions 必須命名。我們在 App Engine 中適當命名主要處理常式 `root()`，說明其用途 (根應用程式處理常式)。由於是 Cloud Function，使用該名稱就比較沒有意義。我們將部署名為 `visitme` 的 Cloud 函式，因此請將該名稱也做為 Python 函式的名稱。同樣地，在第 4 和第 5 個單元中，我們也將 Cloud Run 服務命名為 `visitme`。

以下是更新前後的畫面：

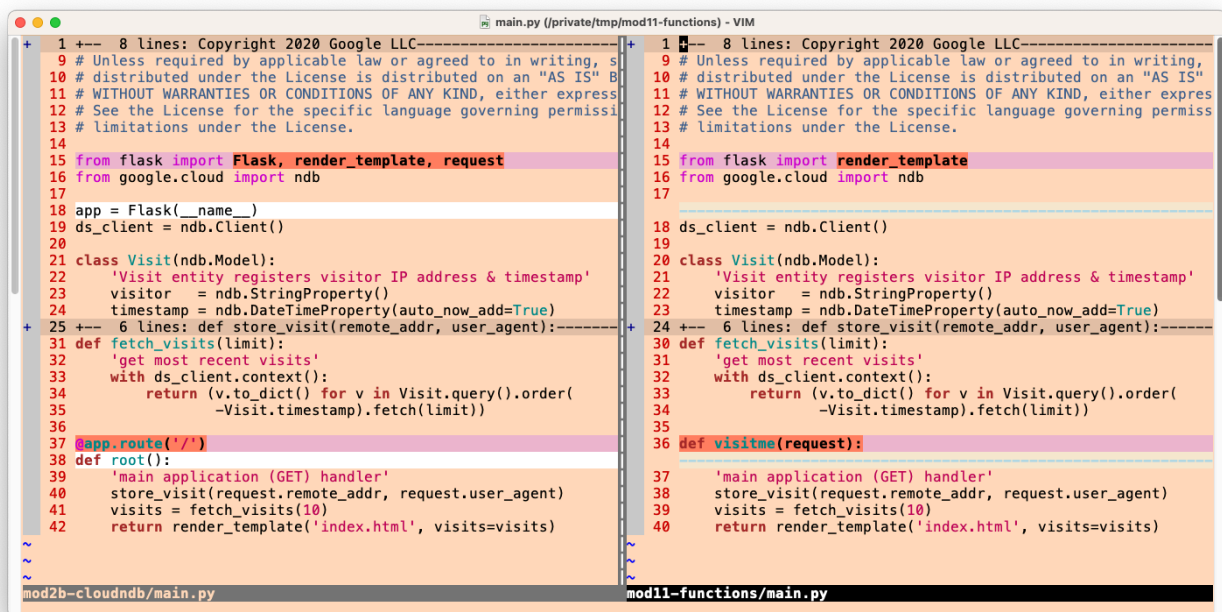
BEFORE:

```
@app.route('/')
def root():
    'main application (GET) handler'
    store_visit(request.remote_addr, request.user_agent)
    visits = fetch_visits(10)
    return render_template('index.html', visits=visits)
```

修改後：

```
def visitme(request):
    'main application (GET) handler'
    store_visit(request.remote_addr, request.user_agent)
    visits = fetch_visits(10)
    return render_template('index.html', visits=visits)
```

所有必要更新到此完成。請注意，所做的變更只會影響應用程式的「基礎架構」程式碼。核心應用程式程式碼不需要任何變更，應用程式功能也不會受到影響。下圖以圖像呈現所做的變更，說明這項重點：



在本機開發及測試

App Engine 有 [dev_appserver.py](#) 本機開發伺服器，Cloud Functions 則有 [Functions Framework](#)。您可以使用這個架構在本機開發及測試。您的程式碼可以部署至 Cloud Functions，但也可以部署至其他運算平台，例如 [Compute Engine](#)、[Cloud Run](#)，甚至支援 [Knative](#) 的地端部署或混合雲系統。如需 Functions Framework 的其他連結，請參閱下文。

步驟 6 · 約 4 分鐘

6. 建構及部署

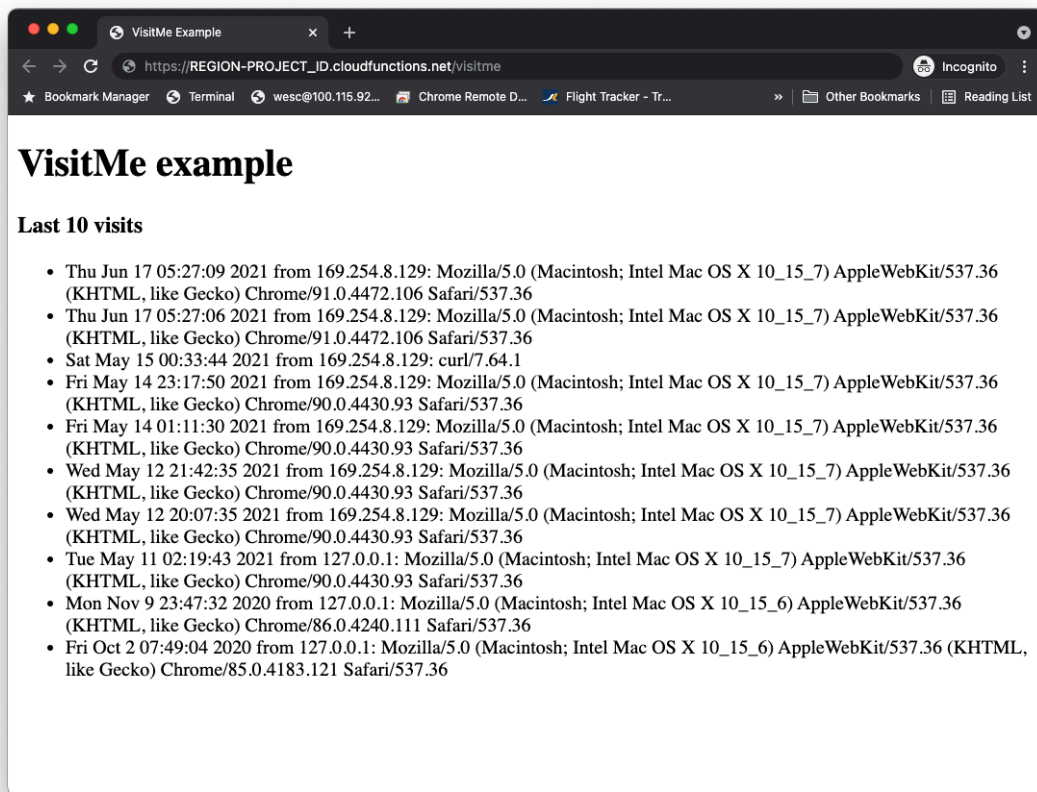
部署至 Cloud Functions 的方式與 App Engine 略有不同。由於 `requirements.txt` 以外沒有使用任何設定檔，因此必須在指令列中指定更多程式碼相關資訊。使用下列指令，以 Python 3.10 部署新的 HTTP 觸發 Cloud 函式：

```
$ gcloud functions deploy visitme --runtime python310 --trigger-http --allow-unauthenticated
```

預期的輸出結果應類似：

```
Deploying function (may take a while - up to 2 minutes)...
For Cloud Build Logs, visit: https://console.cloud.google.com/cloud-build/builds;region=REGION/f5f6fc81-1bb3-4cdb-8bfe?project=PROJECT_ID
Deploying function (may take a while - up to 2 minutes)...done.
availableMemoryMb: 256
buildId: f5f6fc81-1bb3-4cdb-8bfe
buildName: projects/PROJECT_ID/locations/REGION/builds/f5f6fc81-1bb3-4cdb-8bfe
dockerRegistry: CONTAINER_REGISTRY
entryPoint: visitme
httpsTrigger:
  securityLevel: SECURE_OPTIONAL
  url: https://REGION-PROJECT_ID.cloudfunctions.net/visitme
ingressSettings: ALLOW_ALL
labels:
  deployment-tool: cli-gcloud
name: projects/PROJECT_ID/locations/REGION/functions/visitme
runtime: python310
serviceAccountEmail: PROJECT_ID@appspot.gserviceaccount.com
sourceUploadUrl: https://storage.googleapis.com/uploads-853031211983.REGION.cloudfunctions.appspot.com/8c923758-cee8-47ce-8e97-5720a5301c34.zip
status: ACTIVE
timeout: 60s
updateTime: '2022-05-16T18:28:06.153Z'
versionId: '8'
```

函式部署完畢後，請使用部署輸出內容中的網址，造訪您的應用程式。網址格式為 `REGION-PROJECT_ID.cloudfunctions.net/visitme`。輸出內容應與先前部署至 App Engine 時相同：



與本系列中的大多數其他程式碼研究室和影片一樣，基線應用程式功能不會變更。目的是套用一種現代化技術，讓應用程式在較新的基礎架構上運作，但功能與先前完全相同。舉例來說，您可以從舊版 App Engine 舊版服務遷移至替代的 Cloud 獨立產品，或如本教學課程所示，將應用程式遷移至其他 Google Cloud 無伺服器平台。

7. 摘要/清除

恭喜您將這個小型 App Engine 應用程式轉換為 Cloud Function！另一個適用的情境是將大型單體式 App Engine 應用程式拆分成一系列微服務，每個微服務都是 Cloud Function。這是更現代的開發技術，可產生更「隨插即用」的元件 (類似「JAM stack」)。這項技術可讓您混用及比對，並重複使用程式碼，達成兩大目標。此外，這些微服務會持續進行偵錯，因此程式碼穩定性高，整體維護成本也較低。

清除所用資源

完成本程式碼研究室後，您可以[停用第 2 模組的 App Engine 應用程式](#) (暫時或永久)，以免產生費用。App Engine 平台提供[免費配額](#)，只要用量不超過免費方案額度，就不會產生費用。Datastore 也是如此，詳情請參閱 [Cloud Datastore 定價頁面](#)。

部署至 App Engine 和 Cloud Functions 等平台會產生[少許建構和儲存費用](#)。在某些地區，[Cloud Build](#) 和 [Cloud Storage](#) 都有各自的免費配額。建構作業會消耗部分配額。請留意儲存空間用量，盡量減少潛在費用，特別是如果您的所在區域沒有這類免費層級。

很抱歉，Cloud Functions 沒有「停用」功能。備份程式碼，然後刪除函式。您之後隨時可以重新部署，並沿用相同名稱。不過，如果您不打算繼續進行任何其他遷移程式碼研究室，並想徹底刪除所有內容，請[關閉 Cloud 專案](#)。

後續步驟

除了本教學課程外，您還可以查看其他遷移模組，包括將 App Engine 應用程式容器化，以便在 Cloud Run 中執行。請參閱第 4 和第 5 個單元程式碼研究室的連結：

- [單元 4](#)：使用 Docker 遷移至 Cloud Run
- 使用 Docker 將應用程式容器化，以便在 Cloud Run 上執行
- 遷移後，您仍可使用 Python 2。
- [單元 5](#)：使用 Cloud Buildpacks 遷移至 Cloud Run
- 使用 Cloud Buildpacks 將應用程式容器化，以便在 Cloud Run 上執行
- 您不需要瞭解 Docker、容器或 `Dockerfile`。
- 應用程式必須已遷移至 Python 3 (Buildpacks 不支援 Python 2)

其他許多模組的重點，都是向開發人員說明如何從 App Engine 綁定的服務遷移至 Cloud 獨立替代方案：

- [單元 2](#)：從 App Engine `ndb` 遷移至 Cloud NDB
- [單元 7 至 9](#)：從 App Engine 工作佇列推送工作遷移至 Cloud Tasks
- [第 12 至 13 個單元](#)：從 App Engine Memcache 遷移至 Cloud Memorystore
- [單元 15 至 16](#)：從 App Engine Blobstore 遷移至 Cloud Storage
- [單元 18-19](#)：從 App Engine 工作佇列 (提取工作) 遷移至 Cloud Pub/Sub

如果容器化已成為應用程式開發工作流程的一部分，特別是如果工作流程包含 CI/CD (持續整合/持續推送軟體更新或持續部署) 管道，請考慮改用 [Cloud Run](#)，而非 Cloud Functions。請參閱[第 4 模組](#)，瞭解如何使用 Docker 將應用程式容器化；或參閱[第 5 模組](#)，瞭解如何在不使用容器、Docker 知識或 `Dockerfile` 的情況下執行這項操作。無論是考慮使用 Cloud Functions 或 Cloud Run，切換至其他無伺服器平台都是選用做法。我們建議您先考量應用程式和用途的最佳選項，再進行任何變更。

無論您接下來要考慮哪個遷移模組，都可以在 [開放原始碼存放區](#) 存取所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有])。此外，該存放區的 `README` 也提供指引，說明要考慮哪些遷移作業，以及遷移模組的相關「順序」。

8. 其他資源

App Engine 遷移模組程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供第 8 堂課 (START) 和第 9 堂課 (FINISH) 的存放區資料夾連結。您也可以從[所有 App Engine Codelab 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 3
Module 2	code
範本 11	code

線上資源

以下是可能與本教學課程相關的線上資源：

App Engine

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- [App Engine 定價和配額資訊](#)
- [推出第二代 App Engine 平台 \(2018 年\)](#)
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例存放區](#)
- [社群貢獻的遷移範例存放區](#)

Cloud Functions

- [gcloud functions deploy 指令](#)
- [價格資訊](#)
- [Cloud Functions 下一代版本公告](#)
- [第一代和第二代函式的差異](#)
- [函式架構 \(本機開發\) 操作說明](#)、[使用說明文件](#)和[存放區](#)

- [產品頁面](#)
- [說明文件](#)

其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud 「永久免費」 方案](#)
- [Google Cloud SDK](#) (`gcloud` 指令列工具)
- [所有 Google Cloud 說明文件](#)

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- 訂閱 [Google Cloud Tech](#)
- 訂閱 [Google Developers](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
